BÀI LUYỆN TẬP INTERFACE

Bài 1: Xây dựng giao diện `DongVat`

Đề bài

Tạo một **interface** có tên `DongVat` với các phương thức:

- `void keu()`
- `void an()`

Sau đó, tạo hai lớp `Cho` và `Meo` triển khai `DongVat` và định nghĩa cách chúng kêu và ăn.

Giải pháp

```
// Định nghĩa interface DongVat
interface DongVat {
    void keu();  // Phương thức trừu tượng để định nghĩa tiếng kêu của động
vật
    void an();   // Phương thức trừu tượng để định nghĩa cách ăn của động vật
}

// Lớp Cho triển khai interface DongVat
class Cho implements DongVat {
    @Override
    public void keu() {
        System.out.println("Gâu gâu!");
    }

    @Override
    public void an() {
        System.out.println("Chó đang ăn xương...");
    }
}

// Lớp Meo triển khai interface DongVat
class Meo implements DongVat {
    @Override
    public void keu() {
        System.out.println("Meo meo!");
    }

    @Override
    public void an() {
        System.out.println("Mèo đang ăn cá...");
    }
}

// Lớp chạy chương trình chính
public class Main {
    public static void main(String[] args) {
        DongVat cho = new Cho();
```

```
        DongVat meo = new Meo();

        System.out.println("Tiếng kêu của chó:");
        cho.keu();
        cho.an();

        System.out.println("Tiếng kêu của mèo:");
        meo.keu();
        meo.an();
    }
}
```

Phân tích

- **`interface DongVat`** chứa các phương thức trừu tượng `keu()` và `an()`.
- **Lớp `Cho` và `Meo`** triển khai `DongVat` và định nghĩa cụ thể cách kêu và ăn của từng loài.
- **Tính đa hình**: Sử dụng biến `DongVat` để tham chiếu đến `Cho` hoặc `Meo`.

Bài 2: Xây dựng giao diện `PhuongTien`

Đề bài

Tạo một **interface** `PhuongTien` với các phương thức:

- `void khoiDong()`
- `void dungLai()`

Sau đó, tạo hai lớp `OTo` và `XeMay` triển khai `PhuongTien`.

Giải pháp

```
// Định nghĩa interface PhuongTien
interface PhuongTien {
    void khoiDong();
    void dungLai();
}

// Lớp OTo triển khai PhuongTien
class OTo implements PhuongTien {
    @Override
    public void khoiDong() {
        System.out.println("Ô tô đang khởi động...");
    }

    @Override
    public void dungLai() {
        System.out.println("Ô tô đang dừng lại...");
    }
}

// Lớp XeMay triển khai PhuongTien
```

```java
class XeMay implements PhuongTien {
    @Override
    public void khoiDong() {
        System.out.println("Xe máy đang khởi động...");
    }

    @Override
    public void dungLai() {
        System.out.println("Xe máy đang dừng lại...");
    }
}

// Chương trình chính
public class Main {
    public static void main(String[] args) {
        PhuongTien oto = new OTo();
        PhuongTien xemay = new XeMay();

        System.out.println("Hoạt động của ô tô:");
        oto.khoiDong();
        oto.dungLai();

        System.out.println("Hoạt động của xe máy:");
        xemay.khoiDong();
        xemay.dungLai();
    }
}
```

Phân tích

- **Giao diện `PhuongTien`** định nghĩa các phương thức mà mọi phương tiện giao thông cần có.
- **Lớp `OTo` và `XeMay`** triển khai `PhuongTien` để mô tả cách thức hoạt động của từng loại phương tiện.
- **Tính đa hình**: Có thể dùng `PhuongTien` để tham chiếu đến cả `OTo` và `XeMay`.

Bài 3: Xây dựng giao diện `ThanhToan`

Đề bài

Tạo một **interface** `ThanhToan` với phương thức:

- `void xuLyThanhToan(double soTien)`

Tạo hai lớp `ThanhToanTheTinDung` và `ThanhToanPaypal` triển khai `ThanhToan`.

Giải pháp

```java
// Định nghĩa interface ThanhToan
interface ThanhToan {
    void xuLyThanhToan(double soTien);
```

```
}

// Lớp ThanhToanTheTinDung triển khai ThanhToan
class ThanhToanTheTinDung implements ThanhToan {
    @Override
    public void xuLyThanhToan(double soTien) {
        System.out.println("Thanh toán bằng thẻ tín dụng số tiền: $" +
soTien);
    }
}

// Lớp ThanhToanPaypal triển khai ThanhToan
class ThanhToanPaypal implements ThanhToan {
    @Override
    public void xuLyThanhToan(double soTien) {
        System.out.println("Thanh toán bằng PayPal số tiền: $" + soTien);
    }
}

// Chương trình chính
public class Main {
    public static void main(String[] args) {
        ThanhToan thanhToan1 = new ThanhToanTheTinDung();
        ThanhToan thanhToan2 = new ThanhToanPaypal();

        thanhToan1.xuLyThanhToan(500.0);
        thanhToan2.xuLyThanhToan(200.0);
    }
}
```

Phân tích

- **Giao diện `ThanhToan`** định nghĩa phương thức `xuLyThanhToan()`.
- **Lớp `ThanhToanTheTinDung` và `ThanhToanPaypal`** triển khai cách thanh toán riêng.
- **Tính đa hình**: Cho phép chọn phương thức thanh toán linh hoạt.

Bài 4: Xây dựng giao diện `NhanVien`

Đề bài

Tạo một **interface** `NhanVien` với phương thức:

- `double tinhLuong()`

Tạo hai lớp `QuanLy` và `LapTrinhVien` triển khai `NhanVien`.

Giải pháp

```
// Định nghĩa interface NhanVien
interface NhanVien {
    double tinhLuong();
}
```

```java
// Lớp QuanLy triển khai NhanVien
class QuanLy implements NhanVien {
    private double luongCoBan;

    public QuanLy(double luongCoBan) {
        this.luongCoBan = luongCoBan;
    }

    @Override
    public double tinhLuong() {
        return luongCoBan * 1.2; // 20% thưởng
    }
}

// Lớp LapTrinhVien triển khai NhanVien
class LapTrinhVien implements NhanVien {
    private double luongCoBan;

    public LapTrinhVien(double luongCoBan) {
        this.luongCoBan = luongCoBan;
    }

    @Override
    public double tinhLuong() {
        return luongCoBan * 1.1; // 10% thưởng
    }
}

// Chương trình chính
public class Main {
    public static void main(String[] args) {
        NhanVien quanLy = new QuanLy(8000);
        NhanVien lapTrinhVien = new LapTrinhVien(6000);

        System.out.println("Lương Quản Lý: " + quanLy.tinhLuong());
        System.out.println("Lương Lập Trình Viên: " +
lapTrinhVien.tinhLuong());
    }
}
```

Phân tích

- **Giao diện `NhanVien`** định nghĩa phương thức `tinhLuong()`.
- **Lớp `QuanLy` và `LapTrinhVien`** có cách tính lương riêng.
- **Tính đa hình**: Giúp mở rộng hệ thống mà không cần thay đổi mã nguồn cũ.

Tổng kết

- **Interface giúp định nghĩa các hành vi chung** mà các lớp con phải triển khai.
- **Tính đa hình giúp linh hoạt trong việc sử dụng** các đối tượng khác nhau thông qua một interface chung.
- **Mô hình hóa thực tế** tốt hơn so với kế thừa đơn thuần.

Bạn có thể mở rộng thêm các bài toán bằng cách thêm phương thức vào **interface** và thử nghiệm với nhiều lớp khác nhau

BÀI LUYỆN TẬP THÊM

Hướng dẫn luyện tập

- **Nhóm 1**: Rèn luyện kỹ năng sử dụng **interface cơ bản**.
- **Nhóm 2**: Ứng dụng **interface vào các hệ thống thực tế**, sử dụng danh sách đối tượng (`ArrayList`), thêm **tìm kiếm, sắp xếp**.

Nhóm 1: Cơ bản - Định nghĩa và triển khai Interface

1. **Xây dựng Interface `DongVat`**
   - Phương thức: `void keu()`, `void an()`.
   - Lớp `Cho`, `Meo` triển khai `DongVat`.
2. **Xây dựng Interface `PhuongTien`**
   - Phương thức: `void khoiDong()`, `void dungLai()`.
   - Lớp `OTo`, `XeMay` triển khai `PhuongTien`.
3. **Xây dựng Interface `ThanhToan`**
   - Phương thức: `void xuLyThanhToan(double soTien)`.
   - Lớp `ThanhToanTheTinDung`, `ThanhToanPaypal` triển khai `ThanhToan`.
4. **Xây dựng Interface `NhanVien`**
   - Phương thức: `double tinhLuong()`.
   - Lớp `QuanLy`, `LapTrinhVien` triển khai `NhanVien`.
5. **Xây dựng Interface `HinhHoc`**
   - Phương thức: `double tinhDienTich()`, `double tinhChuVi()`.
   - Lớp `HinhChuNhat`, `HinhTron` triển khai `HinhHoc`.
6. **Xây dựng Interface `MayTinhBoTui`**
   - Phương thức: `double cong(double a, double b)`, `double tru(double a, double b)`, ...
   - Lớp `CasioFX500`, `Vinacal500` triển khai `MayTinhBoTui`.
7. **Xây dựng Interface `DieuKhien`**
   - Phương thức: `void bat()`, `void tat()`.
   - Lớp `Tivi`, `DieuHoa` triển khai `DieuKhien`.
8. **Xây dựng Interface `PhatNhac`**
   - Phương thức: `void choiNhac(String baiHat)`.
   - Lớp `Mp3Player`, `Radio` triển khai `PhatNhac`.
9. **Xây dựng Interface `DocGhiFile`**
   - Phương thức: `void docFile(String tenFile)`, `void ghiFile(String tenFile, String noiDung)`.

- Lớp `FileVanBan`, `FileAnh` triển khai `DocGhiFile`.

10. **Xây dựng Interface `DienThoai`**

- Phương thức: `void goiDien(String so)`, `void guiTinNhan(String so, String tinNhan)`.
- Lớp `Android`, `iPhone` triển khai `DienThoai`.

Nhóm 2: Nâng cao - Xây dựng hệ thống với Interface

11. **Quản lý danh sách phương tiện (`PhuongTien`)**

- Sử dụng `ArrayList<PhuongTien>`.
- Thêm, xóa, tìm kiếm phương tiện theo loại.

12. **Quản lý danh sách nhân viên (`NhanVien`)**

- Thêm chức năng tìm nhân viên có lương cao nhất.

13. **Xây dựng hệ thống đăng nhập với Interface `TaiKhoan`**

- `Admin`, `NguoiDungThuong` triển khai `TaiKhoan`.

14. **Xây dựng hệ thống đặt vé (`Ve`)**

- `VeMayBay`, `VeTauHoa` triển khai `Ve`.

15. **Xây dựng hệ thống nhà hàng (`MonAn`)**

- `MonChay`, `MonMan` triển khai `MonAn`.

16. **Xây dựng hệ thống thanh toán điện tử (`ThanhToan`)**

- `ViDienTu`, `TheNganHang` triển khai `ThanhToan`.

17. **Quản lý thư viện (`TaiLieu`)**

- `Sach`, `TapChi` triển khai `TaiLieu`.

18. **Quản lý cửa hàng (`SanPham`)**

- `DienTu`, `ThoiTrang` triển khai `SanPham`.

19. **Xây dựng hệ thống kiểm tra kết quả học tập (`KetQua`)**

- `ThiTracNghiem`, `ThiTuLuan` triển khai `KetQua`.

20. **Quản lý thú cưng (`ThuCung`)**

- `Chim`, `CaCanh` triển khai `ThuCung`.